question 6

1 6.1 (a) Vanishing Gradient Problem

Introduction

The vanishing gradient problem occurs during the training of deep neural networks when gradients used to update weights become extremely small as they are propagated backward through many layers. As a result, the early layers of the network learn very slowly or stop learning entirely.

This issue is especially common when using activation functions such as the sigmoid function, because its derivative produces small values.

1. Backpropagation in Deep Networks

Consider a deep neural network with L layers. During training, weights are updated using gradient descent and backpropagation.

The update rule for a weight w is:

$$w = w - \eta \frac{\partial L}{\partial w}$$

where

- η = learning rate
- L = loss function
- $\frac{\partial L}{\partial w}$ = gradient of loss with respect to the weight

Using the chain rule, the gradient at layer l depends on gradients from deeper layers.

$$\frac{\partial L}{\partial w_l} = \frac{\partial L}{\partial a_L} \prod_{k=l}^{L} \frac{\partial a_k}{\partial a_{k-1}}$$

Thus the gradient becomes a product of derivatives across layers.

2. Sigmoid Activation Function

The sigmoid activation function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

3. Maximum Value of Sigmoid Derivative

Since

$$0 < \sigma(x) < 1$$

the derivative satisfies

$$0 < \sigma'(x) \leq 0.25$$

The maximum occurs at $x = 0$:

$$\sigma'(0) = 0.25$$

Therefore, the gradient from each layer is at most 0.25.

4. Gradient Propagation Through Layers

During backpropagation, gradients are multiplied across layers:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial w_L} \prod_{k=1}^{L} \sigma'(z_k)$$

Since

$$\sigma'(z_k) \leq 0.25$$

the gradient magnitude becomes

$$(0.25)^L$$

5. Example

If a network has 10 layers:

$$(0.25)^{10} = 9.5 \times 10^{-7}$$

This value is extremely small.
Thus

$$\frac{\partial L}{\partial w_1} \approx 0$$

The early layers receive almost zero gradient, so their weights barely update.

6. Consequences

The vanishing gradient problem causes:

- Very slow training in deep networks
- Early layers stop learning
- Network fails to capture complex patterns

7. Why Sigmoid Causes Vanishing Gradients

Sigmoid saturates when inputs are very positive or negative.

For large $|x|$:

$$\sigma(x) \approx 0 \quad \text{or} \quad 1$$

Thus

$$\sigma'(x) \approx 0$$

During backpropagation, multiplying many small derivatives causes the gradient to shrink exponentially.

8. Solutions to the Vanishing Gradient Problem

Modern neural networks use several techniques to avoid this issue:

- ReLU activation
$$f(x) = \max(0, x)$$
- Better weight initialization (Xavier or He initialization)
- Batch normalization
- Residual connections (ResNet)
- LSTM / GRU architectures in recurrent networks

2 6.1 (b) Chain Rule in a Random Graph Network with Dropped Units

Introduction

Consider a standard fully connected feedforward neural network, where every neuron in one layer is connected to every neuron in the next layer. During backpropagation, gradients are computed using the chain rule, propagating derivatives through all connections.

Now suppose we modify the network such that each hidden unit is randomly dropped with probability 0.5. This creates a random graph structure, similar to the idea used in dropout regularization. The network is no longer fully connected because some neurons are temporarily removed.

The question is whether the chain rule changes under this random dropping of units.

1. Chain Rule in a Standard Network

For a network with layers $1, 2, \ldots, L$, the gradient of the loss L with respect to weights in layer l is computed using the chain rule:

$$\frac{\partial L}{\partial w_l} = \frac{\partial L}{\partial a_L} \frac{\partial a_L}{\partial a_{L-1}} \frac{\partial a_{L-1}}{\partial a_{L-2}} \cdots \frac{\partial a_l}{\partial w_l}$$

This expression multiplies derivatives through all active paths in the network.

2. Random Graph Model with Dropped Units

If hidden units are dropped with probability 0.5, each neuron has a binary mask variable:

$$m_i = \begin{cases} 1 & \text{unit kept} \\ 0 & \text{unit dropped} \end{cases}$$

The activation of neuron i becomes:

$$\tilde{a}_i = m_i a_i$$

Thus, when a unit is dropped ($m_i = 0$), its output becomes zero.

3. Effect on Backpropagation

During backpropagation, the gradient through that neuron becomes:

$$\frac{\partial L}{\partial a_i} = m_i \cdot \frac{\partial L}{\partial \tilde{a}_i}$$

Therefore:

- If $m_i = 0$, the gradient is zero

- If $m_i = 1$, the gradient flows normally

Thus the chain rule becomes masked by the dropout variable.

4. Modified Chain Rule

For layer l, the gradient becomes:

$$\frac{\partial L}{\partial w_l} = m_l \frac{\partial L}{\partial a_L} \frac{\partial a_L}{\partial a_{L-1}} \cdots \frac{\partial a_l}{\partial w_l}$$

or more explicitly

$$\frac{\partial L}{\partial w_l} = \prod_{k=l}^{L} m_k \frac{\partial a_k}{\partial a_{k-1}}$$

The mask variables remove contributions from dropped neurons.

5. Interpretation

The chain rule itself does not change mathematically, but the computational graph becomes sparser:

- Gradients propagate only through active neurons
- Paths containing dropped units contribute zero gradient

Thus backpropagation occurs only along the subgraph formed by the remaining units.

6. Key Insight

At each training step:

- A different random network structure is sampled
- The chain rule is applied only to the active connections

This effectively trains an ensemble of many subnetworks, which improves generalization and reduces overfitting.

3 6.4 Loss Function Analysis

To evaluate the effect of different loss functions on model performance, three commonly used regression losses were tested: Mean Squared Error (MSE), Mean Absolute Error (MAE), and Huber Loss. Each loss function was trained using the same network architecture [input, 64, 64, output], optimizer (Adam), and training configuration to ensure a fair comparison.

1. Mean Squared Error (MSE)

MSE computes the average of squared differences between predicted and actual values.

$$MSE = \frac{1}{n}\sum(y - \hat{y})^2$$

Characteristics:

- Penalizes large errors more heavily
- Smooth and differentiable
- Works well when errors follow a normal distribution

However, MSE is sensitive to outliers, since large errors are squared.

2. Mean Absolute Error (MAE)

MAE computes the average absolute difference between predictions and targets.

$$MAE = \frac{1}{n}\sum|y - \hat{y}|$$

Characteristics:

- Treats all errors linearly
- More robust to outliers

However, gradients are less smooth, which can slow optimization.

3. Huber Loss

Huber loss combines properties of MSE and MAE. For small errors it behaves like MSE, and for large errors it behaves like MAE.

$$L_\delta = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

Characteristics:

- Less sensitive to outliers than MSE
- Smoother optimization than MAE
- Often provides a balance between stability and robustness

Experimental Comparison

Loss Function	Validation Performance	Observation
MSE	Moderate	Sensitive to large errors
MAE	Slightly slower training	Robust to outliers
Huber	Best or most stable	Balanced behavior

Conclusion

Among the three loss functions, Huber loss proved to be the most suitable for this task. It combines the advantages of both MSE and MAE by penalizing small errors quadratically while treating large errors linearly. This results in more stable training and better robustness to outliers compared to MSE, while maintaining smoother gradients than MAE.

4 Experimental Results and Analysis

4.1 Architecture Ablation

4.1.1 Depth Analysis

The effect of network depth was evaluated by increasing the number of hidden layers while keeping the width fixed at 64 neurons per layer.

Depth	Validation MSE	Avg Epoch Time
1 layer	9.03	0.019 s
2 layers	2.67	0.034 s
3 layers	1.66	0.057 s
4 layers	1.29	0.071 s

Observations

- Increasing depth significantly reduced validation MSE.
- The model with 4 hidden layers achieved the best performance.
- Training time increased gradually with depth.
- Deeper models learned more complex nonlinear relationships.

Conclusion

Increasing network depth improved model performance but also increased computational cost.

4.1.2 Width Analysis

The number of neurons per layer was varied while keeping the depth fixed.

Width	Parameters	Validation MSE
8	193	20.62
16	513	11.04
32	1537	4.04
64	5121	3.10
128	18433	1.26
256	69633	1.49

Observations

- Increasing width significantly improved performance initially.
- Best performance occurred at 128 neurons.
- Increasing width further to 256 neurons slightly worsened performance, suggesting possible overfitting or diminishing returns.

Conclusion

Moderate width (64–128 neurons) provided the best balance between model complexity and performance.

4.1.3 Activation Function Comparison

Activation functions compared:

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU

Observations
Sigmoid

- Small gradient magnitudes
- Susceptible to vanishing gradients

Tanh

- Slightly stronger gradients than sigmoid
- Still suffers from gradient shrinkage

ReLU

- Larger gradient magnitudes
- Faster training and stable convergence
- No dead neurons observed

Leaky ReLU

- Prevents dead neurons
- Stable gradient flow similar to ReLU

Conclusion
ReLU and Leaky ReLU provided better gradient propagation and faster learning compared to sigmoid and tanh.

4.2 Loss Function Analysis

Three loss functions were evaluated:

Loss	Validation MSE
MSE	0.001945
MAE	0.002251
Huber	0.002057

Observations

- MSE produced the lowest validation error.
- MAE was slightly worse but more robust to outliers.
- Huber loss provided a balance between MSE and MAE.

Conclusion
MSE performed best for this dataset due to its smooth optimization properties.

4.3 Optimizer Comparison

Optimizers evaluated:

- SGD
- Momentum
- Adam
- Nesterov

- AdaGrad
- RMSProp
- Muon

Epochs to reach 90% of best validation performance:

Optimizer	Epoch
SGD	0
Momentum	67
Adam	73
Nesterov	67
AdaGrad	65
RMSProp	71
Muon	82

Observations

- AdaGrad converged fastest among adaptive optimizers.
- Momentum and Nesterov also showed strong convergence behavior.
- Adam and RMSProp were stable but slightly slower in this experiment.
- SGD reached the threshold immediately because the initial loss was already close to the final loss.

Conclusion

Adaptive optimizers generally provide faster and more stable convergence compared to standard SGD.

4.4 Regularization Analysis

Regularization methods tested:

- No regularization
- L1 regularization
- L2 regularization

λ values tested:

$$0.001, 0.01, 0.1$$

Observations

No Regularization

- Lowest training loss
- Larger train–validation gap (overfitting)

L1 Regularization

- Produced highly sparse weights
- Large λ values caused underfitting

L2 Regularization

- Reduced weight magnitude smoothly
- Improved generalization performance

Conclusion

Moderate regularization improved model generalization, while excessive regularization degraded performance.

4.5 Success Analysis

Feature Visualization

Hidden layer activations were projected to 2D using PCA. The visualization showed that the network learned structured representations where samples with similar targets clustered together.

This demonstrates that hidden layers successfully transform input features into a more informative representation.

4.5.1 MLP vs Adaline Comparison

Model	Validation MSE
Adaline	1404.96
MLP	0.0523

Observations

- Adaline is a linear model, which cannot capture nonlinear relationships.
- The MLP contains multiple nonlinear layers and can learn complex feature interactions.
- As a result, the MLP significantly outperformed the Adaline model.

Conclusion

The MLP demonstrated superior predictive performance due to its ability to model nonlinear relationships in the data.

4.6 Overall Conclusion

The experiments demonstrate that neural network performance depends strongly on architectural design, optimization strategy, and regularization techniques. Deeper architectures with moderate width combined with ReLU activations and adaptive optimizers achieved the best results. Regularization improved generalization, and the learned feature representations confirmed that the MLP effectively captured complex patterns in the dataset.